



LAB PRACTICE № 5: POLYMORPHISM, INHERITANCE, INTERFACE

COMP1020: OOP, Algorithms, and Data Structures

Week 05

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your .java file(s) to both the Canvas and CMS Autograder **before the end of the laboratory session** unless the instructor gave a specified deadline.
 - (a) No Canvas submission will get 0 point (of course).
 - (b) **No CMS Autograder submission will get 20% deduction.**
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 - (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 - (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 - (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

Learning Outcome

Topics: Polymorphism, Overriding and Overloading in Java.

Outcome: Acquire the skill of implementing and using polymorphic programming structures, overriding, and overloading.

CMS Autograder

Direct access to this CMS lab is followed: <https://www.cmsvinuni.online>

Note: Due to the current limitation of the Autograder, you must comment out all the `package` declaration before submission.

Problem 01: Products Order (40 pts)

Write a Java program and name it `ProductOrder.java`: You are given a sequence of product orders consisting of two types: Regular Orders and Discounted Orders. Each product has three properties:

- A Product ID (integer)
- A Product Name (String)
- A Price (double)

Each order consists of a Product ID and Quantity to be purchased. If the order includes a discount, the discount percentage is also provided.

Your task is to compute the total price of an order.

Design: use method overloading

```
1  import java.util.Scanner;
2
3  class Order {
4      private int[] productIds;
5      private String[] productNames;
6      private double[] productPrices;
7
8      // Constructor
9      public Order(int numProducts, Scanner scanner) {
10         productIds = new int[numProducts];
11         productNames = new String[numProducts];
12         productPrices = new double[numProducts];
13
14         System.out.println("Enter product details (ID, Name, Price):")↵
15         ;
16         for (int i = 0; i < numProducts; i++) {
17             productIds[i] = scanner.nextInt();
18             productNames[i] = scanner.next();
19             productPrices[i] = scanner.nextDouble();
20         }
21     }
22 }
```

```

19     }
20 }
21
22 // TODO BY STUDENT: Implement a helper method to return the index ←
    of a product ID in the array
23 private int getProductIndex(int productId) {
24     // Loop through productIds and return index if found, ←
        otherwise return -1
25 }
26
27
28 public void placeOrder(int productId, int quantity) {
29     // TODO BY STUDENT: Implement the method to place an order for ←
        a single product without discount, can be done with the ←
        following steps:
30     // Find product index using getProductIndex
31     // If valid, calculate total price and display order details
32     // If invalid (index == -1), display error message "Invalid ←
        Product ID"
33
34
35     System.out.println("Order placed: " + quantity + " x " + ←
        productNames[index] + " | Total: $" + totalPrice);
36 }
37
38 // Overloading
39 public void placeOrder(int productId, int quantity, double ←
    discountPercent) {
40     // TODO BY STUDENT: Implement the method to place an order for ←
        a single product with discount, can be done with the ←
        following steps:
41     // Find product index using getProductIndex
42     // If valid, calculate total price after discount and display ←
        order details
43     // If invalid (index == -1), display error message "Invalid ←
        Product ID"
44
45     System.out.println("Order placed: " + quantity + " x " + ←
        productNames[index] + " | Total after discount: $" + ←
        totalPrice);
46 }
47 }
48
49 public class ProductOrder {
50     public static void main(String[] args) {
51         Scanner scanner = new Scanner(System.in);
52

```

```

53         // Input number of products
54         int numProducts = scanner.nextInt();
55         Order order = new Order(numProducts, scanner);
56
57         // Choose order type
58         System.out.println("Enter order type (1 = Without Discount, 2 ↵
           = With Discount): ");
59         int orderType = scanner.nextInt();
60
61         if (orderType == 1) {
62             int productId = scanner.nextInt();
63             int quantity = scanner.nextInt();
64             order.placeOrder(productId, quantity);
65         } else if (orderType == 2) {
66             int productId = scanner.nextInt();
67             int quantity = scanner.nextInt();
68             double discountPercent = scanner.nextDouble();
69             order.placeOrder(productId, quantity, discountPercent);
70         } else {
71             System.out.println("Error: Invalid order type");
72         }
73
74         scanner.close();
75     }
76 }

```

Input

- Line 1: contains a positive integer n ($1 \leq n \leq 100$) represent number of product category in the store.
- N lines, each containing: Product ID (integer), Product Name (String), Price (double)
- Order type
- One line contain the order information based on order type

Output The total price of the order

Example See Figure 1, 2

Problem 02: Objects moving avoid collision (60 pts)

Recall: interface - An **interface** is a completely "abstract class" that is used to group related methods with empty bodies: Usage example:

```

1 // Interface
2 interface Animal {
3     public void animalSound(); // interface method (does not have body)
4     public void sleep(); // interface method (does not have body)

```

```
3
1001 Headphones 400
1002 Bag 150
1003 Table 240
Enter order type (1 = Without Discount, 2 = With Discount):
2
1003 2 30
Order placed: 2 x Table | Total after discount: $336.0
```

Figure 1: Example testcase of Problem 01

```
3
111 Chair 35
112 Table 60
113 Computer 2000
Enter order type (1 = Without Discount, 2 = With Discount):
2
115 10 30
Invalid Product ID
```

Figure 2: Example testcase of Problem 01

```
5 }
6
7 // Pig "implements" the Animal interface
8 class Pig implements Animal {
9     public void animalSound() {
10         // The body of animalSound() is provided here
11         System.out.println("The pig says: wee wee");
12     }
13     public void sleep() {
14         // The body of sleep() is provided here
15         System.out.println("Zzz");
16     }
17 }
18
19 class Main {
20     public static void main(String[] args) {
21         Pig myPig = new Pig(); // Create a Pig object
22         myPig.animalSound();
23         myPig.sleep();
24     }
25 }
```

FYI: Type Casting in Java: Type casting is when you convert a variable from one data type to another. In Java, there are two types of casting: implicit (automatic) and explicit (manual). When we're working with inheritance, we often store objects as their parent type or interface type, but then need to

access their specific properties.

- Implicit Casting (Widening): Widening casting is done automatically when passing a smaller size type to a larger size type.

```
1 // A dog is also an animal (no explicit cast needed)
2 Dog myDog = new Dog();
3 Animal myAnimal = myDog; // Implicit casting
```

- Explicit Casting (Narrowing): Narrowing casting must be done manually by placing the type in parentheses () in front of the value:

```
1 // We need to explicitly tell Java this Animal is actually a Dog
2 Animal someAnimal = new Dog();
3 Dog someDog = (Dog) someAnimal; // Explicit casting with (Dog)
```

- Real-world example: Imagine you have a parking lot (general container) with different vehicles:

```
1 // Our Vehicle hierarchy
2 class Vehicle {
3     public void move() { System.out.println("Moving..."); }
4 }
5
6 class Car extends Vehicle {
7     public void honk() { System.out.println("Beep!"); }
8 }
9
10 class Bicycle extends Vehicle {
11     public void ringBell() { System.out.println("Ring!"); }
12 }
13
14 // Using the vehicles
15 public class ParkingLot {
16     public static void main(String[] args) {
17         // A parking spot can hold any vehicle (implicit casting)
18         Vehicle parkingSpot1 = new Car();
19         Vehicle parkingSpot2 = new Bicycle();
20
21         // Both can move since that is a Vehicle method
22         parkingSpot1.move();
23         parkingSpot2.move();
24
25         // But to use specific methods, we need explicit casting
26         // parkingSpot1.honk(); // Error! Vehicle does not have ↵
27         honk()
```

```

28         // We need to cast first (tell Java this is actually a Car↵
           )
29         Car myCar = (Car) parkingSpot1;
30         myCar.honk(); // Now works!
31
32         // We can also check the type first to avoid errors by ↵
           using instanceof
33         if (parkingSpot2 instanceof Bicycle) {
34             Bicycle myBike = (Bicycle) parkingSpot2;
35             myBike.ringBell();
36         }
37     }
38 }

```

Task: Write a Java Programming and name it MovingObjectAvoidCollision.java: Perform a sequence of operations in a 2D simulation moving object platform. There are 2 kinds of objects:

- Rectangle: represented by (x, y, LX, LY, direction, speed) in which (x,y) is the coordinate of the center and the length of the horizontal/vertical segments are respectively 2*LX and 2*LY; direction can be "L" - left, "R" - right, "U" - up, "D" - down, and speed represents the displacement of the center then the object performs one move following its direction.
- Circle: represented by (x, y, r, direction, speed) in which (x,y) is the coordinate of the center and r is the radius of the circle; direction can be "L" - left, "R" - right, "U" - up, "D" - down, and speed represents the displacement of the center then the object performs one move following its direction.

Two objects collide each other if they have common points.

The operations have 5 types:

- create RECTANGLE <ID> <x> <y> <LX> <LY> <DIRECTION> <SPEED>: create a rectangle object with center (x,y), segments are 2*LX, 2*LY, the direction is <DIRECTION> and speed is <SPEED>
- create CIRCLE <ID> <x> <y> <r> <DIRECTION> <SPEED>: create a circle object with center (x,y), radius <r>, direction <DIRECTION> and speed <SPEED>
- move <ID>: if object ID = <ID> exists, then it performs a move (if after moving, it does not collide with any other objects).
- change_direction <ID> <DIRECTION> <SPEED>: if object ID = <ID> exists, then change the direction and the speed of the object <ID>
- view_position <ID>: if the object ID = <ID> exists, then display x and y (separated by a SPACE character) - the current coordinate of the center of the object <ID>. Otherwise, display NULL

Design: use method overriding

```

1
2 import java.io.BufferedReader;
3 import java.io.FileReader;
4 import java.io.InputStreamReader;
5 import java.io.PrintWriter;

```

```

6  import java.util.*;
7
8  interface MoveObject{
9      public void move();
10     public void moveOpposite();
11     public boolean collideIfMove(MoveObject o);
12     public boolean collide(MoveObject o);
13     public String getID();
14     public String toString();
15
16 }
17 class BaseObject{
18     public String ID;
19     public String direction;
20     public int speed;
21
22     public BaseObject(String direction, int speed) {
23         this.direction = direction;
24         this.speed = speed;
25     }
26
27     public BaseObject(String ID, String direction, int speed) {
28         this.ID = ID;
29         this.direction = direction;
30         this.speed = speed;
31     }
32     public void changeDirection(String direction, int speed){
33         this.direction = direction; this.speed = speed;
34     }
35 }
36 class MRectangle extends BaseObject implements MoveObject{
37     public int x,y;
38     public int LX, LY;// length of horizontal and vertical segments
39
40
41     public MRectangle(String ID, String direction, int speed, int x, ←
        int y, int LX, int LY) {
42         super(ID, direction, speed);
43         this.x = x;
44         this.y = y;
45         this.LX = LX;
46         this.LY = LY;
47     }
48
49     @Override
50     public void move() {
51         // TODO BY STUDENT

```

```

52     }
53
54     @Override
55     public void moveOpposite() {
56         // TODO BY STUDENT
57     }
58
59     @Override
60     public boolean collideIfMove(MoveObject o) {
61         if(o instanceof MCircle){
62             MCircle c = (MCircle)o;
63             // TODO by STUDENT
64
65         }else if(o instanceof MRectangle){
66             MRectangle r = (MRectangle) o;
67             // TODO by STUDENT
68         }
69         return false;
70     }
71
72     @Override
73     public boolean collide(MoveObject o) {
74         if(o instanceof MCircle){
75             MCircle c = (MCircle)o;
76             // TODO by STUDENT
77         }else if(o instanceof MRectangle){
78             MRectangle r = (MRectangle) o;
79             // TODO by STUDENT
80         }
81         return false;
82     }
83
84     @Override
85     public String getID() {
86         return ID;
87     }
88     @Override
89     public String toString(){
90         return x + " " + y;
91     }
92 }
93
94 class MCircle extends BaseObject implements MoveObject{
95     public int x,y;
96     public int radius;
97     @Override
98     public void move() {

```

```

99         // TODO by STUDENT
100     }
101
102     @Override
103     public void moveOpposite() {
104         // TODO BY STUDENT
105     }
106
107     @Override
108     public boolean collideIfMove(MoveObject o) {
109         // return true if the current object move and collide with o
110         if(o instanceof MCircle){
111             MCircle c = (MCircle) o;
112             // TODO by STUDENT
113         }else if(o instanceof MRectangle){
114             MRectangle r = (MRectangle) o;
115             // TODO by STUDENT
116         }
117         return false;
118     }
119
120     @Override
121     public boolean collide(MoveObject o) {
122         if(o instanceof MCircle){
123             MCircle c = (MCircle) o;
124             // TODO by STUDENT
125         }else if(o instanceof MRectangle){
126             MRectangle r = (MRectangle) o;
127             return r.collide(this);
128         }
129         return false;
130     }
131
132     @Override
133     public String getID() {
134         return ID;
135     }
136
137     public MCircle(String ID, String direction, int speed, int x, int y, int radius) {
138         super(ID, direction, speed);
139         this.x = x;
140         this.y = y;
141         this.radius = radius;
142     }
143
144     @Override
145     public String toString(){

```

```

145         return x + " " + y;
146     }
147
148 }
149 public class MovingObjectAvoidCollision {
150     ArrayList<MoveObject> objects = new ArrayList<>();
151     MoveObject findByID(String ID) {
152         for(MoveObject o: objects)
153             if(o.getID().equals(ID)) return o;
154         return null;
155     }
156     public void run(){
157         try{
158             BufferedReader in = new BufferedReader(new ↵
159                 InputStreamReader(System.in));
160
161             String line; String[] s;
162             while(true){
163                 line = in.readLine();
164                 if(line.equals("#")) break;
165                 s = line.split(" ");
166                 if(s[0].equals("create")){
167                     String TYPE = s[1];
168                     String ID = s[2];
169                     if(TYPE.equals("RECTANGLE")){
170                         int x = Integer.parseInt(s[3]);
171                         int y = Integer.parseInt(s[4]);
172                         int LX = Integer.parseInt(s[5]);
173                         int LY = Integer.parseInt(s[6]);
174                         String direction = s[7];
175                         int speed = Integer.parseInt(s[8]);
176                         MRectangle r = new MRectangle(ID,direction,↵
177                             speed,x,y,LX,LY);
178                         boolean ok = true;
179                         for(MoveObject o: objects){
180                             if(r.collide(o)){ ok = false; break;}
181                         }
182                         if(ok)
183                             objects.add(r);
184                     }else if(TYPE.equals("CIRCLE")){
185                         int x = Integer.parseInt(s[3]);
186                         int y = Integer.parseInt(s[4]);
187                         int radius = Integer.parseInt(s[5]);
188                         String direction = s[6];
189                         int speed = Integer.parseInt(s[7]);
190                         MCircle c = new MCircle(ID,direction,speed,x,y↵

```

```

        ,radius);
190         boolean ok = true;
191         for(MoveObject o: objects){
192             if(c.collide(o)){ ok = false; break;}
193         }
194         if(ok)
195             objects.add(c);
196
197     }
198     }else if(s[0].equals("move")){
199         String ID = s[1];
200         MoveObject o = findById(ID);//mID20bject.get(ID);
201         if(o != null) {
202             BaseObject bo = (BaseObject) o;
203             boolean okMove = true;
204             for (MoveObject oo : objects)
205                 if (oo != o) {
206                     if (o.collideIfMove(oo)) {
207                         okMove = false;
208                         break;
209                     }
210                 }
211             if (okMove) o.move();
212         }
213     }else if(s[0].equals("change_direction")){
214         String ID = s[1];
215         String direction = s[2]; int speed = Integer.parseInt(s[3]);
216         MoveObject o = findById(ID);
217         BaseObject bo = (BaseObject) o;
218         if(bo != null) {
219             bo.changeDirection(direction, speed);
220         }
221     }else if(s[0].equals("view_position")){
222         String ID = s[1];
223         MoveObject o = findById(ID);
224         if(o == null) System.out.println("NULL");
225         else System.out.println(o.toString());
226     }
227 }
228 }catch (Exception e){ e.printStackTrace();}
229 }
230 public static void main(String[] args){
231     Main app = new Main();
232     app.run();
233 }
234 }

```

```
create RECTANGLE R01 0 0 1 2 L 2
create CIRCLE C01 5 5 2 R 1
move R01
change_direction R01 U 3
change_direction C01 L 1
move C01
view_position R01
-2 0
move C02
view_position C01
4 5
move R01
view_position C01
4 5
create RECTANGLE R02 0 0 3 4 R 1
view_position R02
NULL
| #
```

Figure 3: Example testcase of Problem 02

Input

- Sequence of lines, each line contains an operation described above. The input is terminated with a line containing #

Output

- Write the result of the view_position operation.

Example See Figure 3